



UNIVERSIDAD NACIONAL DEL SUR

PRINCIPIOS DE COMPUTADORAS II

Proyecto N°1

Space Invaders - Diseño Orientado a Objetos-

Integrante:

Antonella Malaspina

Destinatarios:

Profesor: Dr. Juan Miguel Bajo

Asistente: Ing. Alejandro Zambrano

Ayudante: Ing. Jorge Iván Lopez Perez

Bahía Blanca, Buenos Aires

Fecha de reentrega:

24 de noviembre de 2025

Índice

1. Introducción al Diseño del Sistema	2
2. Descripción General del Sistema	2
3. Diagrama UML de Clases	3
3.1. Relaciones Principales	3
3.2. Descripción de las Clases	4
3.2.1. Dibujable_Abstracto	4
3.2.2. Colisionable	5
3.2.3. Movil	6
3.2.4. Alien	7
3.3. Clases de Control del Sistema	12
3.3.1. GameWidget	12
3.3.2. GameLogic	12
4. Interacciones entre Clases	15
5. Justificación del Diseño	15
6. Conclusión	15

1. Introducción al Diseño del Sistema

El presente informe describe el diseño orientado a objetos (POO) y la arquitectura modular implementados para el desarrollo del videojuego Space Invaders en C++ utilizando Qt.

La estructura del sistema se fundamenta en un diagrama UML de clases que modela las entidades, sus comportamientos y las relaciones entre los distintos módulos. Este diagrama no solo proporciona una representación estructurada de cómo se distribuyen las responsabilidades y el flujo de datos, sino que también garantiza la reutilización de código, la claridad y la escalabilidad del proyecto.

El diseño del juego se basa en principios de POO como la herencia, la abstracción y el encapsulamiento. Esto se logra definiendo clases base clave como `Dibujable_Abstracto` (para la representación gráfica), `Colisionable` (para la gestión de impactos) y `yMovil` (para los objetos que se mueven), cuyas funcionalidades se combinan a través de herencia múltiple en entidades esenciales como `Nave`, `Proyectil Alien` y `Barrera`.

La reutilización se extiende a los enemigos: las clases `Pulpo`, `Cangrejo` y `Calamar` se especializan a partir de la clase base `Alien`, permitiendo modelar distintos enemigos sin duplicar código.

Finalmente, la lógica central del juego reside en la clase `GameLogic`, que administra objetos, movimiento y colisiones, mientras que `GameWidget` actúa como la capa de interacción con el usuario y gestiona la actualización gráfica a través de Qt. El objetivo es analizar esta arquitectura robusta y modular en detalle.

2. Descripción General del Sistema

El videojuego consiste en una nave controlada por el jugador, capaz de moverse horizontalmente y disparar proyectiles mediante la barra espaciadora. Con la tecla `ESC` el usuario puede pausar la partida.

Los enemigos están organizados en hileras con los siguientes puntajes:

- **Pulpo:** otorga 10 puntos
- **Cangrejo:** otorga 20 puntos
- **Calamar:** otorga 30 puntos
- **Nave enemiga:** otorga 50 puntos (aparece aleatoriamente)

Las barreras otorgan protección parcial al jugador, degradándose a medida que reciben impactos. La lógica central del juego determina los movimientos, colisiones, creación de enemigos y actualización del puntaje.

-
- **Dependencia:** `GameWidget` depende de `Tiempo_proceso_Qpainter` para la renderización

3.2. Descripción de las Clases

Clases Base Abstractas

Las siguientes clases definen las interfaces fundamentales del sistema:

3.2.1. `Dibujable_Abstracto`

Rol: Es una de las clases abstracta que define los atributos geométricos y la interfaz gráfica mínima de cualquier objeto que pueda dibujarse en pantalla.

Responsabilidad: Estandarizar la estructura de los objetos gráficos del juego (posición, tamaño y vida) y garantizar la existencia de un método de dibujo.

Atributos:

- `vida: int` - cantidad de vidas del objeto
- `x, y: float` - posición del objeto en el espacio 2D
- `ancho, alto: float` - dimensiones del sprite

Métodos clave:

- `get_x(): float, get_y(): float` - obtener posición
- `get_ancho(): float, get_alto(): float` - obtener dimensiones
- `Dibujar(QPainter&): void*` - método puro virtual para renderizar el objeto

Relaciones: Es la base de todos los objetos visibles. Implementada por: `Alien`, `Nave`, `Proyectil`, `Barrera`, `Nave_enemiga`.

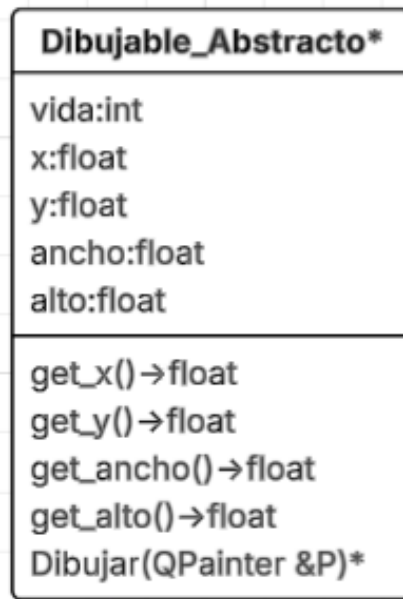


Figura 2: Clase Dibujable_Abstracto

3.2.2. Colisionable

Rol: Define la interfaz de detección de colisiones mediante áreas rectangulares. Es otra clase abstracta, donde su método abstracto es el `get_area()`*.

Responsabilidad: Brindar el mecanismo común para obtener áreas de colisión (`QRectF`) y evaluar choques entre objetos mediante intersección de rectángulos.

Atributos:

- `colisionado: bool` - indica si el objeto ya colisionó

Métodos:

- `get_area(): QRectF*` - método puro virtual que retorna el área de colisión
- `CheckCollision(const colisionable* const *bool)` - usa `intersects()` entre rectángulos
- `get_colision(): bool` - consulta el estado de colisión
- `set_colision(bool): void` - establece el estado de colisión

Relaciones: Todos los objetos colisionables derivan de esta clase. Implementada por: `Alien`, `Nave`, `Proyectil`, `Nave_enemiga`, `Barrera`.

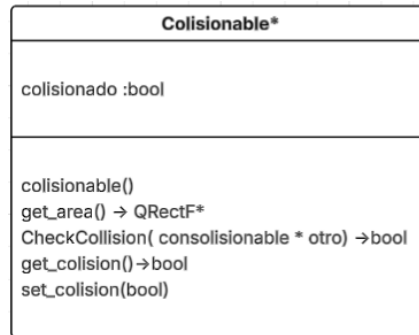


Figura 3: Clase Colisionable

3.2.3. Movil

Rol: Clase abstracta que define comportamiento de movimiento para objetos dinámicos.

Responsabilidad: Proporcionar una representación independiente para velocidad y movimiento, separada de las responsabilidades de dibujo y colisión.

Atributos:

- `Vx: float` - velocidad horizontal
- `Vy: float` - velocidad vertical

Métodos:

- `movimiento(): void*` - método puro virtual para actualizar la posición
- `set_Vx(float): void` - establece velocidad horizontal
- `set_Vy(float): void` - establece velocidad vertical

Relaciones: La lógica de movimiento se unifica bajo esta clase. Implementada por: `Nave`, `Alien`, `Proyectil`, `Nave_enemiga`.

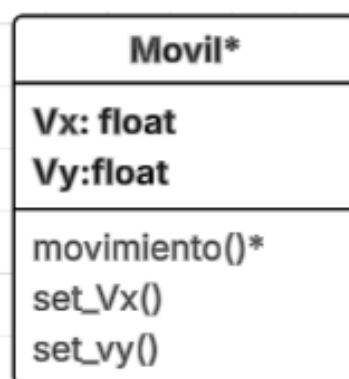


Figura 4: Clase Movil

Clases de Entidades del Juego

3.2.4. Alien

Clase base abstracta que define el comportamiento común de todos los enemigos.

Rol: Clase base que representa a cualquier enemigo del juego.

Responsabilidad: Establecer la estructura general de los aliens: posición, vida, velocidad y forma de dibujar/disparar.

Atributos:

- Heredados de `Dibujable_Abstracto`: vida, x, y, ancho, alto
- Heredados de `Movil`: Vx, Vy
- Heredados de `Colisionable`: colisionado
- `expresion_alien`: int - identificador del tipo de alien

Métodos:

- `Dibujar(QPainter&): void*` - renderiza el alien (virtual)
- `movimiento(): void*` - actualiza posición según velocidades
- `Disparar(): proyectil*` - genera proyectil enemigo
- `get_area(): QRectF*` - retorna área de colisión

Relaciones:

- Herencia simple hacia: `Calamar`, `Cangrejo`, `Pulpo`
- Composición: Es administrado por `GameLogic` y `Enemigos`

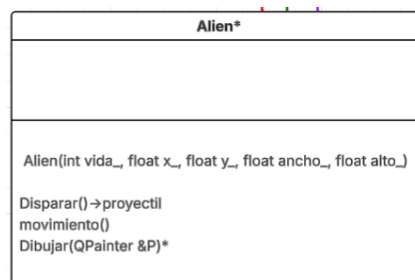


Figura 5: Clase Alien

Clases derivadas: Pulpo, Cangrejo, Calamar Cada tipo de enemigo hereda de `Alien` y redefine sus características principales:

Pulpo:

- Valor: 10 puntos por eliminación
- Posición: Filas inferiores
- Sprite específico con animación característica

Cangrejo:

- Valor: 20 puntos por eliminación
- Posición: Filas intermedias
- Sprite específico con animación característica

Calamar:

- Valor: 30 puntos por eliminación
- Posición: Filas superiores
- Sprite específico con animación característica

Cada clase sobrescribe el método `Dibujar()` para renderizar su sprite específico y puede ajustar valores de velocidad o comportamiento según el nivel de dificultad.

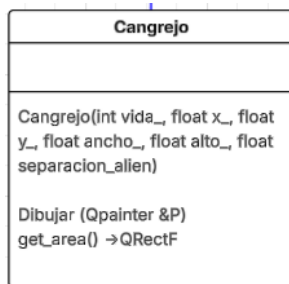


Figura 6: Cangrejo

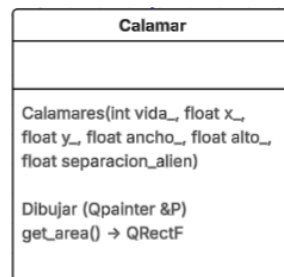


Figura 7: Calamar

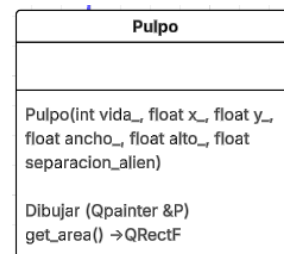


Figura 8: Pulpo

Nave

Rol: Representa al jugador, su sprite, movimiento y sistema de vidas.

Responsabilidad: Controlarse horizontalmente mediante input del usuario y disparar proyectiles ascendentes.

Atributos:

- Hereda de `Dibujable_Abstracto`: vida, x, y, ancho, alto
- Hereda de `Movil`: Vx, Vy
- Hereda de `Colisionable`: colisionado
- `vidas_iniciales`: int = 3 - cantidad de vidas al inicio

Métodos:

- `movimiento()`: void* - actualiza posición horizontal
- `Dibujar(QPainter&)`: void* - renderiza la nave
- `get_area()`: QRectF* - retorna área de colisión
- `Disparar()`: proyectil* - genera proyectil del jugador
- `recibir_danio()`: void - reduce vidas al recibir impacto

- `esta_destruida(): bool` - verifica si perdió todas las vidas

Relaciones: Usada y administrada por `GameLogic`. Genera objetos `Proyectil`.

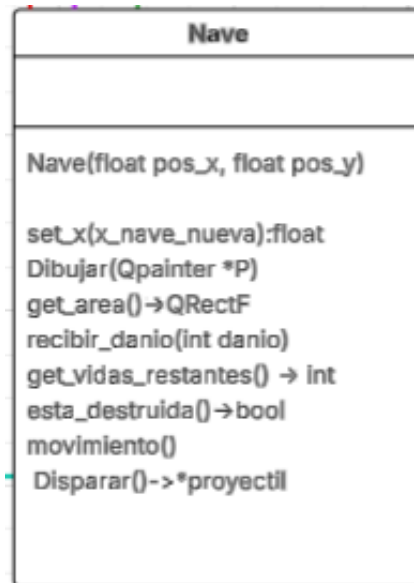


Figura 9: Clase Nave del jugador

Proyectil

Rol: Representa los disparos tanto del jugador como de los enemigos.

Responsabilidad: Moverse verticalmente, impactar objetos y dibujarse en pantalla. Auto-eliminarse al salir de los límites de la pantalla.

Atributos:

- Hereda de `Dibujable_Abstracto`: `x`, `y`, `ancho`, `alto`
- Hereda de `Movil`: `Vx`, `Vy`
- Hereda de `Colisionable`: `colisionado`
- `velocidad: float` - rapidez del movimiento vertical

Métodos:

- `movimiento(): void*` - actualiza posición vertical usando `Vy`
- `Dibujar(QPainter&): void*` - renderiza el proyectil
- `get_area(): QRectF*` - retorna área de colisión

Características:

- Los proyectiles del jugador se mueven hacia arriba ($Vy < 0$)
- Los proyectiles enemigos se mueven hacia abajo ($Vy > 0$)
- Dimensiones pequeñas para facilitar la jugabilidad

Relaciones: Instanciado por: `Alien::Disparar()` y `Nave::Disparar()`. Administrado por `GameLogic`.

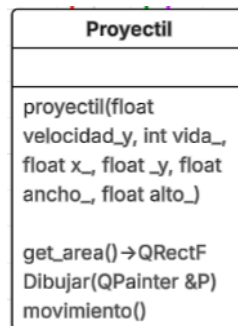


Figura 10: Clase Proyectoil

Barrera

Rol: Bloque de protección que absorbe proyectiles y se decolora con cada impacto.

Responsabilidad: Representar una estructura degradable entre el jugador y los aliens, proporcionando protección temporal.

Atributos:

- Hereda de `Dibujable_Abstracto`: vida, x, y, ancho, alto
- Hereda de `Colisionable`: colisionado

Métodos:

- `Dibujar(QPainter&): void*` - renderiza la barrera con estado visual según daño
- `get_area(): QRectF*` - retorna área de colisión
- `recibir_danio(): void` - reduce resistencia y actualiza apariencia

Características:

- Se posiciona estratégicamente entre la nave y los aliens
- Cambia su apariencia visual al recibir daño
- Se destruye completamente al perder toda su resistencia
- No es móvil (no hereda de `Movil`)

Relaciones: Creada, dibujada y removida por `GameLogic`.

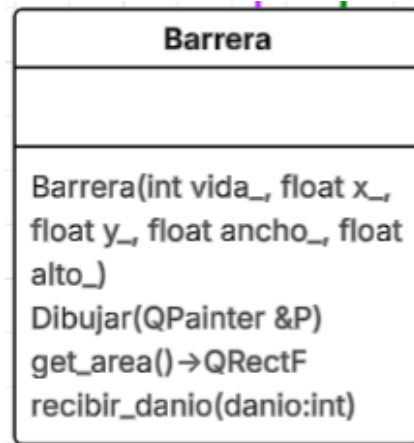


Figura 11: Clase Barrera

Nave Enemiga

Rol: Enemigo especial que aparece ocasionalmente en la parte superior.

Responsabilidad: Moverse horizontalmente, desaparecer y otorgar mayor puntaje.

Atributos:

- Dirección
- Velocidad independiente
- Vale 100 puntos

Métodos:

- movimiento()
- fuera_de_pantalla()
- Dibujar()
- get_area()

Relaciones: Generada aleatoriamente en GameLogic.

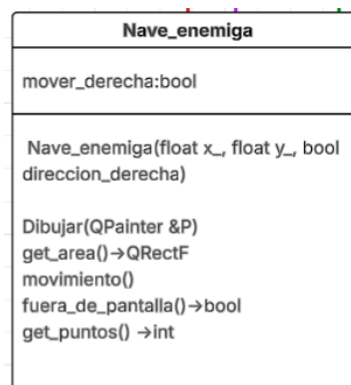


Figura 12: Clase de la nave enemiga

3.3. Clases de Control del Sistema

3.3.1. GameWidget

Rol: Clase principal que coordina la interfaz gráfica y el flujo del juego.

Responsabilidades principales:

- Gestionar el ciclo de renderizado mediante `Tiempo_proceso_Qpainter`
- Coordinar la lógica del juego a través de `GameLogic`
- Administrar la creación de enemigos mediante `Enemigos`
- Manejar eventos de teclado y mouse del usuario
- Procesar el loop principal del juego

Atributos principales:

- `timer_principal: Qpainter` - temporizador para actualización de pantalla
- Referencias a `GameLogic` y `Enemigos`
- `pausa: bool` - estado de pausa del juego
- `juego_iniciado: bool` - indica si el juego comenzó

Métodos clave:

- `paintEvent(): void` - renderiza la escena
- `keyPressEvent(QKeyEvent*): void` - maneja teclas presionadas
- `keyReleaseEvent(QKeyEvent*): void` - maneja teclas liberadas
- `actualizar_juego(): void` - actualiza el estado del juego
- `pausar_bool(): bool` - consulta estado de pausa

3.3.2. GameLogic

Rol (central del sistema): Es el núcleo del videojuego, actúa como el núcleo del funcionamiento del juego, gestionando la creación, actualización, dibujo, destrucción y sincronización de todos los elementos del juego: la nave del jugador, enemigos, proyectiles, barreras y la nave bonus. Esta clase define un motor de juego completo, responsable del ciclo principal: entrada → actualización lógica → renderizado → control de estado, manteniendo el flujo coherente y autónomo de una partida completa.

Responsabilidades principales:

- Inicializar el juego y configurar el estado inicial
- Crear y administrar todos los objetos del juego
- Mover entidades según sus velocidades
- Detectar y resolver colisiones entre objetos

-
- Generar enemigos y proyectiles dinámicamente
 - Controlar el menú principal y el sistema de pausa
 - Gestionar puntaje, vidas y condiciones de victoria/derrota
 - Administrar la nave bonus y su generación aleatoria

Atributos principales:

- nave_principal: Nave* - puntero a la nave del jugador
- lista_barreras: QLists<barrera*> - lista de barreras activas
- lista_gameovers: QLists<dibujables*> - elementos de pantalla de game over
- lista_proyectil: QLists<proyectil*> - proyectiles activos en el juego
- proyectil_nave: QLists<proyectil*> - proyectiles de la nave del jugador
- lista_a_dibujar: QLists<Dibujable*> - todos los objetos a renderizar
- lista_a_mover: QLists<Movil*> - todos los objetos móviles
- nave_bonus: Nave_enemiga* - nave especial que otorga puntos extra
- intervalo_disparo_alien: int - control de frecuencia de disparo enemigo
- margen_derecha: int, margen_izquierda: int - límites de movimiento
- contacto_barrera_alien: int - contador de impactos
- cantidad_alien_caidos: int - aliens eliminados
- direccion_alien_derecha: bool - dirección actual del grupo de aliens
- pausa: bool - estado de pausa del juego

Métodos clave:

- inicializar_juego(): void - configura el estado inicial
- actualizar(): void - ciclo principal del juego
- Dibujar(Qpainter&): void - renderiza todos los objetos
- Verificar_colisiones(): void - detecta y procesa todas las colisiones
- Mover_enemigos(): void - actualiza posiciones de aliens
- Disparar_alien(): void - genera proyectiles enemigos
- Generar_nave_bonus(): void - crea la nave especial aleatoriamente
- Pressbutton(Qt::key): void - procesa tecla presionada
- Releasebutton(Qt::key): void - procesa tecla liberada
- Colision_proyectil(proyectil_proyectil*): void - resuelve colisiones de proyectiles
- limpiar_listas(): void - elimina objetos destruidos
- pausar_bool(): bool - retorna el estado de pausa

Relaciones: Agrega, mueve y elimina todos los objetos del juego. Se comunica con GameWidget para actualizar pantalla y con Enemigos para la gestión de aliens.



Figura 13: Diagrama de la clase GameLogic

4. Interacciones entre Clases

El ciclo de ejecución del juego sigue el siguiente flujo:

1. Lectura del input del usuario (movimiento y disparo)
2. Actualización del estado de la nave
3. Movimiento de los enemigos
4. Actualización de proyectiles
5. Detección de colisiones:
 - Proyectil–Alien
 - Proyectil–Barrera
 - Alien–Barrera
 - Alien–Nave
6. Actualización del puntaje
7. Renderizado de la escena

5. Justificación del Diseño

El diseño adoptado sigue los principios de la Programación Orientada a Objetos:

- **Modularidad:** Cada entidad del juego está encapsulada en su propia clase
- **Herencia:** Evita duplicación de código para enemigos que comparten comportamiento
- **Agregación y composición:** `GameLogic` controla el conjunto de objetos sin crear dependencias rígidas
- **Escalabilidad:** Permite agregar nuevos tipos de enemigos o modificaciones sin alterar la estructura base
- **Mantenibilidad:** El sistema puede ser extendido y corregido fácilmente

6. Conclusión

El diagrama UML presentado permite visualizar de manera clara la arquitectura del videojuego Space Invaders. La estructura de clases elegida ofrece un diseño robusto, escalable y coherente con los principios de ingeniería de software. Esto facilita futuras expansiones como nuevos niveles, animaciones adicionales o enemigos alternativos.